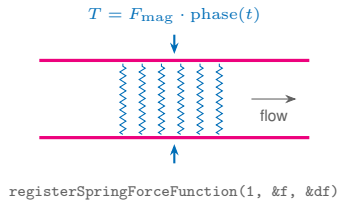
CUSTOM SPRING FORCES AND ACTIVE PUMPING

Writing your own spring force function, applying forces to a structure, and driving a heart tube three ways



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 9b

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. write a spring force function of your own (a nonlinear spring, a muscle, an applied force), register it with IBAMR's force generator under a force-function index, and select it per spring from the `.spring` file;
2. use the spring's parameters for whatever your law needs, and change them in time from an update function, since the force function itself never sees the time;
3. drive a structure with an applied force rather than with target points, and say when each is the better choice;
4. read and run the heart-tube pump example (IB/11) in its five actuation modes: suction pumping by target points or by force, peristalsis by a travelling force, a travelling stiffness, or travelling targets;
5. know the other ways to put a force on a structure (a Lagrangian force strategy, a uniform body force, an Eulerian forcing function) and recognize the mistakes that make an applied force do nothing or too much.

> Chapter 9a moved target points and changed springs; this chapter changes the law of the spring. Chapter 36 (Part VI) builds the two-point valveless pump on the same ideas; chapter 11 reads the forces out.

HOW THIS CHAPTER WAS CHECKED

- The force-function interface is quoted from the IBAMR 0.19.0 headers (`IBSpringForceFunctions.h`, `IBStandardForceGen.h`) and from the force generator's source; the 2013 interface the course first taught is shown next to it (slide 8).
- The example IB/11 (heart-tube pump) is the lab's 2014 `Forced-tube-pump` code ported to IBAMR 0.16.0 and 0.19.0, with the five actuation modes of the 2014 lecture and code selectable from the input file; it builds with 0 warnings, four modes run two periods with the shipped settings, and the fifth (the stiffness wave) stops at 0.64 s when the walls stick, which is reported as such (slides 37, 28). The figures are from those runs.
- The 2014 lecture's code had an undefined variable, a loop that added packets that had left the tube, a rest-length slot reused as a phase, and a structure-order dependency; each is named on the slide where it occurs and fixed in IB/11. Its three peristalsis modes, run as written, aborted before 0.8 s; the slide "What can go wrong (2)" shows why, and the shipped parameters were chosen from runs that complete.
- The Python generator writes files identical to the MATLAB one (Octave 8.4 against Python 3.12, 27 files, token for token).

> Slide numbers refer to this deck; chapter numbers (3, 5, 6, 7, 8a, 8b, 9a, 11, 15, 26, 36) to the other chapters of this series.

WHERE TO LOOK THINGS UP

- The interface: `include/ibamr/IBSpringForceFunctions.h` defines the two function-pointer types and the default Hookean law, with its documentation; `IBStandardForceGen.h` declares `registerSpringForceFunction`. The 2013 slides called the first file `IBSpringFunction.h`, which never existed under that name.
- The upstream example: `examples/IB/explicit/ex3` registers its own linear spring law (slide 9); the 2013 and 2014 lectures both pointed to it.
- The force generator: `src/IB/IBStandardForceGen.cpp`, the loop of chapter 8b that calls your function once per spring per force evaluation.
- The examples of this series: IB/11 (this chapter), with its `main.cpp` (the force functions and their registration), five update functions, five input files and a README with the measured results; IB/06 (chapter 9a) for the update-function pattern.
- The IBAMR FAQ, https://github.com/IBAMR/IBAMR/wiki/FAQ, and the class documentation, https://ibamr.github.io/docs.

WHY NEW SPRING FUNCTIONS?

From the October 2013 lecture, *Creating your own spring function in IBAMR*:

- "If you want a nonlinear spring or a spring that behaves like a muscle, you may need to create a new spring function."
- "IBAMR has a way to do this by registering your function and giving your type of spring a number that can be used in `.spring` files."
- "Set up a spring function and register it with the `IBStandardForceGen` object that is created in the main routine. If you use function index 0 when registering your spring function, your function will become the 'default' spring function."
- "Note that you can set the spring function number in the `.spring` file. E.g. different spring functions can be assigned to different links."

> A "spring" in IBAMR is any force between two points that acts along the line joining them and depends on their distance and on numbers you store with the spring. Hooke's law is the default; a muscle, a cable that goes slack, a nonlinear tissue law, or a force you apply from outside (this chapter's pump) are all springs with a different function. "The force will be applied along the (normalized) fiber connecting the two node points" (2015).

THE INTERFACE, IN IBAMR 0.19.0

```
// include/ibamr/IBSpringForceFunctions.h (0.19.0)
// A spring force function returns the (undirected) "tension" T from
//   R              the distance between the "master" and "slave" nodes
//   params         the numbers on the spring's line of the .spring file
//   lag_mastr_idx  the Lagrangian index of the master node
//   lag_slave_idx  the Lagrangian index of the slave node
using SpringForceFcnPtr =
    double (*)(double R, const double* params, int lag_mastr_idx, int
  lag_slave_idx);
// The derivative dT/dR, same arguments
using SpringForceDerivFcnPtr =
    double (*)(double R, const double* params, int lag_mastr_idx, int
  lag_slave_idx);

// The default, force function index 0: a Hookean spring
inline double default_spring_force(double R, const double* params, int, int)
{
    return params[0] * (R - params[1]);   // stiffness * (length - rest
  length)
}
inline double default_spring_force_deriv(double, const double* params, int,
  int)
{
    return params[0];
}
```

Your function returns one number, the tension T: positive pulls the two points together, negative pushes them apart. The generator turns it into the two forces (chapter 8b's listing):

$$\mathbf{F}_{\text{master}} = T\,\frac{\mathbf{X}_{\text{slave}} - \mathbf{X}_{\text{master}}}{R},$$

$$\mathbf{F}_{\text{slave}} = -\mathbf{F}_{\text{master}}.$$

If R is below machine precision the generator skips the spring, so your function never divides by zero. The default is Hooke's law with the stiffness in `params[0]` and the rest length in `params[1]`; chapter 6 explains why the file stiffness is $K/\Delta s$.

THE INTERFACE (2): PARAMETERS, INDICES, DERIVATIVE

```
// include/ibamr/IBStandardForceGen.h: give the function a number
void registerSpringForceFunction(int force_fcn_index,
                                 const SpringForceFcnPtr spring_force_fcn_ptr,
                                 const SpringForceDerivFcnPtr spring_force_deriv_fcn_ptr = nullptr);
```

- `params` is the spring's own line of the file after the two indices: as many numbers as you wrote, in that order (chapter 9a); the default reads two. They are stored per spring, so two springs with the same index can carry different numbers.
- The two indices let a law depend on *which* spring it is (a stiffness that varies along a fibre, a different rest length at the ends) without extra parameters.
- The derivative dT/dR is used only when IBAMR needs the Jacobian of the forces (implicit time stepping, chapter 15). Pass `nullptr` and the generator estimates it by a centred finite difference with step $\max(R, 1)\,\epsilon^{1/3}$; pass the true derivative and it is exact and cheap.
- The force function index is an `int` of your choosing; the constructor of `IBStandardForceGen` registers the default under 0 before you get the object.

THE 2013 INTERFACE, FOR READING OLD CODE

The 2013 lecture showed the then-current default from `IBSpringForceFunctions.h`:

```
inline void
default_linear_spring_force(double F[NDIM], const double D[NDIM],
    const double& stf, const double& rst,
    const int& lag_mastr_idx, const int& lag_slave_idx)
{
    // Compute the distance between the "master" and "slave" nodes.
    double R_sq = 0.0;
    for (int d = 0; d < NDIM; ++d) R_sq += D[d]*D[d];
    const double R = sqrt(R_sq);
    // Compute the force applied to the "master" node.
    const double& kappa = stf;
    const double& R0 = rst;
    if (R > std::numeric_limits<double>::epsilon())
        for (int d = 0; d < NDIM; ++d)
            F[d] = kappa*(1.0-(R0/R))*D[d];
    else
        for (int d = 0; d < NDIM; ++d) F[d] = kappa*D[d];
    return;
}
// and in the constructor (2013), then as now:
IBStandardForceGen::IBStandardForceGen()
{ registerSpringForceFunction(0, &default_spring_force, &
  default_spring_force_deriv); }
```

	2013	2015 onward, 0.19.0
returns	nothing; fills `F[NDIM]`, the force vector on the master	`double`, the tension T; the generator makes the vectors
distance	`D[NDIM]`, the displacement vector; the function computes R	`R`, already computed
parameters	two named arguments, `stf` and `rst`	`const double* params`, any number of them
derivative	none	a second function, or `nullptr`
header name in the slides	`IBSpring Function.h`	`IBSpringForce Functions.h`

Porting a 2013 function: compute T from R and `params` (for the law above, $T = \kappa(R - R_0)$, which is `kappa*(1 - R0/R)*R`), drop the loops, and add a derivative. The zero-distance guard is now inside the generator.

THE UPSTREAM EXAMPLE: EX3 OVERRIDES THE DEFAULT

```
// examples/IB/explicit/ex3/example.cpp (0.19.0): a linear spring
  written in strain
inline double
linear_spring_force(double R, const double* params, int /*
  lag_mastr_idx*/, int /*lag_slave_idx*/)
{
    const double stf = params[0];
    const double rst = params[1];
    return stf * (R / rst - 1.0);
}
inline double
linear_spring_force_deriv(double /*R*/, const double* params, int /*
  lag_mastr_idx*/, int /*lag_slave_idx*/)
{
    const double stf = params[0];
    const double rst = params[1];
    return stf / rst;
}
...
Pointer<IBStandardForceGen> ib_force_fcn = new IBStandardForceGen();
ib_force_fcn->registerSpringForceFunction(0, &linear_spring_force, &
  linear_spring_force_deriv);
ib_method_ops->registerIBLagrangianForceFunction(ib_force_fcn);
```

"Note that the first argument to `registerSpringForceFunction()` is the force function index. In the example, the default linear spring function (force function index 0) is overridden. If you wanted to make a different function that can be used with the default, assign your new function some other integer index." (2013, 2014)

- ex3's law is Hooke's law written in strain: $T = \kappa\,(R/r_0 - 1) = (\kappa/r_0)(R - r_0)$. The same spring as the default, but its file stiffness means force per unit *strain*, which is chapter 6's K, not $K/\Delta s$: overriding index 0 changes what every number in every `.spring` file means. Say so in the generator when you do it.
- The derivative is given exactly, κ/r_0.
- ex3 is the filament of chapter 13 (penalty IB, with mass); its force law is independent of that.

THE FIFTH COLUMN OF THE `.spring` FILE

"By default, springs in `.spring` files get force function index 0. The `.spring` file format is `i j spring_constant rest_length force_fcn_idx`, in which (i, j) is the pair of points connected by a given spring. You can mix-and-match different force function indices in the same `.spring` file." (2013, 2014)

```
41
0 41 5.0000000000000000e-01 0.0000000000000000e
  +00 1
1 42 5.0000000000000000e-01 0.0000000000000000e
  +00 1
...
```

IB/11's `vp_aforce_512.spring`: point i on the top wall to point $i + 41$ below it, two parameters (here a force magnitude, $F_{\mathrm{mag}} = 0.25$, and a phase, not a stiffness and a rest length), force function 1.

In 0.19.0 (`IBStandardInitializer.cpp`):

- the index is read if present and defaults to 0; a line with only four numbers is a default spring;
- `uniform_spring_force_fcn_idx` in the structure's input block gives every spring of that structure the same index, like the other `uniform_*` keys of chapter 6;
- the parameters are stored per spring as a vector; the force function receives a pointer to that spring's vector, so a line may carry three or five numbers if your law reads them (all lines of one structure should carry the same count);
- an index that was never registered has no function behind it: slide 35 shows what happens.

The generator writes the index with the line (`'%d %d %1.16e %1.16e %d\n'`); chapter 6's format table has the column.

WRITING YOUR OWN: THE RECIPE

Four steps, from the 2015 lecture *Creating spring force functions* (Alex Hoover) and the 2014 one:

1. Declare the function "in `main.C`, in the function prototypes section before or after the main call", with the interface of slide 6: `inline double my_force(double R, const double* params, int lag_mastr_idx, int lag_slave_idx)`. Here "`R` is the distance, `params` holds the two parameters (usually the spring constant and the resting length, but this can be changed)".
2. Decide what the parameters mean and write them into the `.spring` file in the generator, with the index in the fifth column.
3. Register it "in the 'Configure the IB solver' section":

```
Pointer<IBStandardForceGen> ib_force_fcn = new IBStandardForceGen();
ib_force_fcn->registerSpringForceFunction(1, &my_force, &my_force_deriv);   // the new line
ib_method_ops->registerIBLagrangianForceFunction(ib_force_fcn);
```

4. If the law changes in time, write an update function (chapter 9a) that sets a parameter each step; the force function reads it.

> The force function has no clock. Its arguments are R, the parameters and the two indices; the time is not among them. Anything time-dependent must be put into a parameter by an update function before the step (the "phase" of the next slides), or read from a global that `main.cpp` sets, which is uglier. This is the single most common surprise in writing one.

A SINUSOIDAL FORCE (2015)

"Using this structure, let's create a simple sinusoidal force, $F = F_{\text{mag}} \sin(ft)$. The force magnitude F_{mag} is stored in `params[0]`. The value of $\sin(ft)$ is stored in `params[1]` and will be updated in the `update_springs.C` routine."

```
// main.C (2015), the force function
inline double frequency_spring_force(double R, const double* params,
   int lag_mastr_idx, int lag_slave_idx)
{
    const double F = params[0];
    const double phase_fun = params[1];
    return F*phase_fun;
}
```

```
// update_springs.C (2015), "the important lines"
double freq = 1;
double theta = freq*current_time*2*pi;
const int lag_idx = node_idx->getLagrangianIndex();
double& phase = spring_spec->getParameters()[0][1];
if (muscle_lag_idxs.first <= lag_idx && lag_idx < muscle_lag_idxs.
  second)
{
    phase = sin(theta);
}
```

Read together:

- the update function runs before each step and writes $\sin(2\pi ft)$ into the second parameter of every spring of the "muscle" structure (through a reference, correctly: `double& phase`);
- the force function runs inside the step, as often as the forces are evaluated, and returns F_{mag} times whatever is in that slot: a tension that oscillates between $\pm F_{\text{mag}}$, pulling the two points together and pushing them apart in turn, whatever their distance;
- the rest-length slot is reused as the phase. Nothing in IBAMR objects; only your own reading of the file must remember it.

The 2015 public-folder copy of this example (`14-YourOwnForceEx`) shipped `main.C` and `update_springs_freq.C` (frequency 0.86 Hz) but no input or structure files; IB/11's `FORCE_VP` mode is this force on a real structure.

REGISTERING IT, AND THE DERIVATIVE

```
// main.C (2015), "Configure the IB solver"; "new addition
  highlighted in red"
Pointer<IBStandardInitializer> ib_initializer = new
  IBStandardInitializer(
    "IBStandardInitializer", app_initializer->getComponentDatabase("
  IBStandardInitializer"));
ib_method_ops->registerLInitStrategy(ib_initializer);
Pointer<IBStandardForceGen> ib_force_fcn = new IBStandardForceGen();
ib_force_fcn->registerSpringForceFunction(1, &frequency_spring_force,
   &frequency_spring_force_deriv);  // <- the new line
ib_method_ops->registerIBLagrangianForceFunction(ib_force_fcn);
```

```
// the derivative the 2014 and 2015 lectures wrote
inline double frequency_spring_force_deriv(double R, const double*
  params, int lag_mastr_idx, int lag_slave_idx)
{
    return 0;
}
```

The public-folder slides said “new addition highlighted in red”; the highlighting did not survive the export. The new line is the one with `registerSpringForceFunction(1, ...)`: index 1, so the default spring (index 0) stays available for the structure’s own springs.

About that derivative:

- it is correct: $T = F_{\mathrm{mag}} \cdot \mathrm{phase}$ does not depend on R, so $dT/dR = 0$. The lectures did not say why it is zero, and a reader might copy it for a law that does depend on R, where it would be wrong;
- for an explicit run (all the examples here) the derivative is never called; for an implicit one it enters the Jacobian, and a wrong derivative slows or breaks the Newton solve without changing the physics;
- when in doubt, pass `nullptr`: IBAMR then differentiates T numerically (`IBStandardForceGen.cpp`: a centred difference with a step of $\max(R, 1)\,\epsilon^{1/3}$).

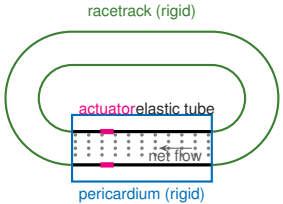
IB/11 registers one function, `applied_force`, under both indices 1 and 2 with the zero derivative; the two `.spring` files use one index each.

THE HEART-TUBE PUMP (IB/11): WHAT IT IS

The 2014 lecture *Designing your own spring force function: examples for peristalsis and dynamic suction pumping*: “We will start with a simple model of dynamic suction pumping where a section of target points drives the contraction.” Its parts, each a structure:

- `heart_tube`: “the elastic section of the tube”, two straight walls 0.42 m long with springs, beams, and target points on ten rigid points at each end;
- `heart_race`: “the racetrack section”, the rigid rest of the loop (target points);
- `heart_peri`: “the pericardium model”, a rigid box around the elastic section (target points);
- `actuator_top` and `actuator_bot`: “target points that drive the motion”, a 4 cm piece of each wall a quarter of the way along;
- `markers`: “passive boundary points that move with the fluid”, 121 of them inside the tube.

$\mathrm{Wo} = 10$ at $f = 1$ Hz (the input sets μ from it); the course ran it for 10 s, the series for 2 s.



The loop, redrawn from the 2014 slide: a 1 m box; tube diameter 0.1 m; inner radius of the bends 0.1 m; pericardium 0.2 m across; actuator 0.04 m long, centred a quarter of the way along the elastic section. Chapter 36 derives the geometry for the two-point pump. The arrow is the net flow along the elastic section measured in both suction modes (chapter 29).

THE GENERATOR AND THE INPUT: PARTS, NUMBERS, ORDER

```
// input2d (mode TARGET_VP): the driving pieces first
ACTUATION = "TARGET_VP"
FRE = 1.0                          // pumping frequency (Hz)
WO  = 10.0                         // Womersley number
MU  = 2.0*PI*FRE*RHO*D*D/(4.0*WO*WO)   // 0.157 N s/m^2
...
IBStandardInitializer {
   max_levels = MAX_LEVELS
   structure_names = "actuator_top_512", "actuator_bot_512",
                     "heart_tube_512", "heart_peri_512",
                     "heart_race_512", "markers_512"
   ...
}
```

"Note on order: in `input2d` the actuator (one of `actuator_top/bot`, `vp_aforce`, `peri_aforce`) must be first under `structure_names`" (2014): the update functions ask for structure 0 (and 1). Five input files, one per mode, differ only in `ACTUATION` and this list; `make_inputs.py` writes the other four from `input2d`.

The driven points of the actuators are copies of points of the tube: each actuator structure holds its own vertices at the same places, and the tube's points there stay ordinary elastic points. A tether or an applied force on the copy moves the fluid of those grid cells, and the tube's wall goes with it. This is why the counts of the tube (860) do not change between modes, and why the input lists the driving pieces as separate structures.

Stiffnesses: $K = 2$, $EI = 10^{-2}$, $\kappa = 2$ (file values 2048, 1.07×10^7, 2048); $F_{\text{mag}} = 0.25$ for the suction force and 1 for the travelling one (the 2014 code's 4, "my best guess at a reasonable applied force", closes the tube: slide 28). The fluid: $\rho = 1000$, $\mu = 0.157$ from $\mathrm{Wo} = 10$ at $f = 1$ Hz.

THE GENERATOR: PARTS AND NUMBERS

structure	files	points
`heart_tube_512`	vertex, spring (858), beam (856), target (40)	860
`heart_tube_stiff_512`	the same, beams $\times 10$ (mode 3)	860
`heart_race_512`	vertex, target	3436
`heart_peri_512`	vertex, target	1066
`markers_512`	vertex only	121
`actuator_top/bot_512`	vertex, target	2×41
`pperi_top/bot_512`	vertex, target	2×308
`vp_aforce_512`	vertex, spring (41, index 1)	82
`peri_aforce_512`	vertex, spring (328, index 2)	656
`peri_kwave_512`	vertex, spring (328, index 0)	656

"The MATLAB code generates each of the boundaries with parameters specified towards the beginning of the code. Note that $N = 512$ assumes the fluid is solved on a 512×512 grid. If L is the length of the domain, then the distance between the boundary points must be close to $L/(2N)$." (2014)

The Python generator `generate_mesh2d.py` is a line-by-line port of the 2014 MATLAB one (shipped too, with a few edits) and writes the same 27 files token for token. The geometry its parameters set: a 1 m box; tube diameter 0.1 m; straight sections 0.42 m (elastic part 0.4 m plus ten rigid points at each end); bends of inner radius 0.1 m; pericardium 0.2 m across; points $\Delta s = L/(2N) = 1/1024$ m apart, half a grid cell.

MODE 1: DYNAMIC SUCTION PUMPING WITH TARGET POINTS

```
// update_target_point_positions.cpp (IB/11), the motion
const double squeeze = (pump::amp / 2.0) * (1.0 + std::sin(2.0 *
  M_PI * freq * current_time - M_PI / 2.0));
...
if (actuator_top_idxs.first <= lag_idx && lag_idx <
  actuator_top_idxs.second)
    X_target[1] = -pump::R2 - squeeze;   // the top wall, at y = -R2,
   moves down
if (actuator_bot_idxs.first <= lag_idx && lag_idx <
  actuator_bot_idxs.second)
    X_target[1] = -pump::R1 + squeeze;   // the bottom wall, at y =
  -R1, moves up
```

amp $= 0.8\times$ diameter$/2$: at full squeeze the walls are 20% of the diameter apart. The targets are set to a position computed from the time (chapter 9a's second form), so the motion is the same whatever DT is and survives a restart.

"How to turn on VP with target points: in main.C, uncomment update_target_point_positions and comment out update_springs_vp_aforce / update_springs_peri_aforce." (2014) In IB/11 that is ACTUATION = "TARGET_VP", and main.cpp calls the matching function:

```
if (actuation == "TARGET_VP")
    update_target_point_positions(patch_hierarchy,
  l_data_manager, loop_time, dt, freq);
else if (actuation == "FORCE_VP")
    update_springs_vp_aforce(patch_hierarchy, l_data_manager,
   loop_time, dt, freq);
...
```

Why it pumps: the squeeze is not in the middle of the elastic section, so the two sides of the tube respond differently to it (one is nearer the rigid racetrack); the asymmetry makes a net flow round the loop. This is Liebau's valveless pumping, which chapter 36 studies with two pumps.

“THINGS YOU MIGHT WANT TO CHANGE”

The 2014 lecture’s list, kept, with where each change lives in IB/11:

Change	2014	IB/11
the diameter	`generate_mesh2d.m` and `update_target_point_positions.C`	the generator and `pump_geometry.h` (one header the update functions share)
removing the racetrack or pericardium	remove them from `structure_names`	the same
Wo, by changing the viscosity	`input2d`	`WO` in the input; μ follows
the pumping frequency	`input2d` and the update file	`FRE` in the input only (passed to the update functions)
the location or width of the actuator	`generate_mesh2d.m`	the generator (`La`, `Ca`)
material properties	`generate_mesh2d.m`	the generator (`kappa_spring`, `kappa_beam`, `kappa_target`)
the force magnitude (“which will change the amplitude of contraction”)	`generate_mesh2d.m`	the generator (`Fmag`, written into the `.spring` files)
the width and speed of the peristaltic packet	hard-coded in the update files	`PERI_WIDTH`, `PERI_SPEED` in the input

Two numbers are repeated between files by design and must agree: the diameter and the point spacing (generator and header), and the actuator’s index range (the generator’s `Na1`, `Na2` and the structure order in the input). Chapter 8a’s rule: change one thing, rerun the generator, check the counts in the log.

MODE 2: THE SAME PUMP BY AN APPLIED FORCE

"Part 2: create your own force. Goal: we want to drive the motion of the tube by adding an external force to the boundary. Application 1: valveless suction pumping with applied force. Let's create a simple applied sinusoidal force of the form $F = F_{\mathrm{mag}}(1 + \sin(2\pi f t - \pi/2))$." (2014)

```
// main.cpp (IB/11): the force function, registered under indices
   1 and 2
inline double applied_force(double /*R*/, const double* params,
  int, int)
{
    const double F = params[0];          // Fmag, from the .
  spring file
    const double phase_fun = params[1];  // set every step by the
   update function
    return F * phase_fun;
}
```

```
// update_springs_vp_aforce.cpp (IB/11): the phase for this step
const double phase = 0.5 * (1.0 + std::sin(2.0 * M_PI * freq *
  current_time - M_PI / 2.0));
...
for (std::vector<double>& spring : spring_spec->getParameters())
    spring[1] = phase;
```

The structure `vp_aforce_512`: the actuator's $41 + 41$ points, with a "spring" from each top point i to the bottom point $i + 41$ under it, parameters F_{mag} and phase, index 1 (slide 10). "Note that we are modifying the phase and not the F_{mag}."

- the tension $F_{\mathrm{mag}} \cdot \mathrm{phase}$ pulls each pair of wall points together: a force across the tube, zero when the phase is zero;
- the 2014 code's phase is $\frac{1}{2}(1 + \sin)$, between 0 and 1, the lecture's $1 + \sin$ between 0 and 2; IB/11 uses the code's;
- unlike mode 1, the walls are not told where to go: "depending on the choice of F_{mag}, the contraction may be too much or too little. You will need to tune it as you change parameters." The 2014 value, 4, closed the tube at 0.24 s and the pinch slid along the tube; 2 closed it at 0.32 s, 1 at 0.44 s, 0.5 in its second squeeze; IB/11 ships 0.25 (slides 28, 29).

"The pieces are the same as the valveless pump driven by target points. A sinusoidal force is applied in the same region."

MODE 3: PERISTALSIS BY A TRAVELLING FORCE

"Application 2: peristalsis with applied force. We will model the dynamics of the force as a Gaussian wave packet, $\text{phase} = \exp(-(x - x_c)^2/w^2)$, where x_c is the center of the Gaussian, w is the width. x_c travels with ct, where c is the wave speed and t is time. We will determine the time since the beginning of the pulse using time − floor(time/period)·period; the pulse number is floor(time/period)." (2014, and the June 2014 note)

```
// update_springs_peri_aforce.cpp (IB/11)
const double period = 1.0 / freq;
const double ww = peri_width;                          // width of the
  packet (m): PERI_WIDTH in the input
const double speed = peri_speed * lperi / period;  // wave speed (m/
  s): PERI_SPEED piece lengths per period
const int pulse_number = static_cast<int>(std::floor(current_time /
  period));
const double pulse_time = current_time - pulse_number * period;
const double xc = speed * pulse_time - 0.15 * lperi;
...
const double x = (lag_idx - muscle_idxs.first) * pump::ds;   // the
  spring's position
const double phase = std::exp(-(x - xc) * (x - xc) / (ww * ww));
for (std::vector<double>& spring : spring_spec->getParameters())
  spring[1] = phase;
```

The structure `peri_aforce_512`: the middle 0.32 m of the tube, $328 + 328$ points, springs across the tube with F_{mag}, phase and index 2. One packet per period starts just before the piece and runs along it at `PERI_SPEED` piece lengths per period; where it is, the walls are pulled together; the squeeze travels and pushes the fluid ahead of it.

What the 2014 slides had, and what IB/11 fixes: next slide.

MODE 3 (2): WHAT THE 2014 SLIDES HAD, AND WHAT IB/11 FIXES

- An undefined variable. The lecture's loop over previous pulses used a variable `c` that was never defined (it meant `speed`), and it added the packets of all earlier pulses, which had long left the piece. One packet per period is the same thing, and is what IB/11 does.
- Hard-coded width and speed. The lecture's width and speed were 0.025 m and 1.0 piece length per period, the code's 0.005 m and 1.3, both hard-coded. IB/11 reads them from the input (`PERI_WIDTH`, `PERI_SPEED`) and ships the lecture's width with the code's speed, because the 5 mm packet with $F_{\mathrm{mag}} = 4$ closed the tube completely at the first pinch (slide 37).
- A stiffer tube for this mode. IB/11 runs the force-driven wave on a tube ten times stiffer in bending (`heart_tube_stiff_512`, $EI = 0.1$, at $\Delta t = 2.5 \times 10^{-5}$), because with the 2014 tube the fluid the packet pushes ahead piles up and the slack tube behind the packet collapses and sticks, whatever the force (slides 28, 29).
- A harmless slip. The lecture's slide 33 registered index 2 with the *VP* function; the two functions are identical, so nothing changed.

MODE 4: THE 2014 CODE'S PERISTALSIS, A TRAVELLING STIFFNESS

The lecture described the force version; the code the lab actually ran in 2014 did something cleverer with the *default* spring: `peri_kwave_512.spring` has springs across the tube with stiffness 0 and rest length $(1 - \text{percont}) \times$ diameter (the 2014 code's `percont` $= 0.8$ gave 0.02 m; IB/11 ships 0.5, 0.05 m: slide 37), index 0, and the update function sets each spring's *stiffness* to a travelling Gaussian:

```
// update_springs_peri_kwave.cpp (IB/11)
const double kmax = (kappa_spring / 10.0) / pump::ds;   // 102.4 in
  file units
...
const double stiffness = kmax * std::exp(-(x - xc) * (x - xc) / (ww
  * ww));
for (std::vector<double>& spring : spring_spec->getParameters())
  spring[0] = stiffness;
```

Where the packet is, the springs are stiff and pull the walls toward the rest length; elsewhere they are slack. Hooke's law does the rest. The packet's width and speed are `PERI_WIDTH` and `PERI_SPEED`, as in mode 3, and the same taper at the piece's ends applies.

Force or stiffness?

- an applied force does not know when to stop: with F_{mag} too large the walls meet and cross, and the 2014 `main.C` added an ad-hoc penalty to its peristaltic force (`F*(phase - 1/(R*512/4)^2)`, "a penalty that will hopefully stop the boundaries from getting too close");
- a stiffness wave with a rest length is bounded by construction: the squeeze cannot go past the rest length, and $k_{\max}$ sets how hard it tries. The price is that the contraction amplitude depends on the tube's own stiffness and the fluid, like a muscle's does;
- a target wave (next slide) is exact and bounded, but it is kinematics, not a force: the walls go where they are told whatever the fluid does.

All three are "your own force" in the sense of this chapter; the parameters they change are different slots of the same spring.

MODE 5: PRESCRIBED PERISTALSIS WITH TARGET POINTS

```
// update_target_point_positions_peri.cpp (IB/11)
const double s_ramp = 0.5 / freq;
    // ramp over half a period
const double c_amp = (current_time < s_ramp) ? current_time / s_ramp
   : 1.0;
const double L_taper = 0.05;
    // the wave dies out over 5 cm at each end
auto envelope = [&](double s) { ... raised cosine from 0 to 1 over
  L_taper, mirrored at Lap ... };
...
const double s = (lag_idx - top_idxs.first) * pump::ds;
    // position along the piece (m)
const double wave = std::sin(-2.0 * M_PI * freq * current_time + 2.0
   * M_PI * (s / pump::Lap));
X_target[1] = -pump::R2 - pump::amp * c_amp * envelope(s) * wave;
    // top wall
...
X_target[1] = -pump::R1 + pump::amp * c_amp * envelope(s) * wave;
    // bottom wall
```

The structures `pperi_top/bot_512`: the middle 0.3 m of each wall, every point a target. One sine wavelength fits the piece and travels to the right at f wavelengths per second; the crest narrows the tube to 20%, the trough widens it into the pericardium's gap.

This is the `update_target_point_positions_peri` of the 2014 `Forced-tube-pump` code, which its `main.C` was calling when the folder was archived (with the other three calls commented out), plus the taper. Compared with mode 1 it is a longer piece with a wave instead of a uniform squeeze; compared with modes 3 and 4 it prescribes the shape instead of a force.

- the ramp `c_amp` avoids the impulsive start of chapter 7;
- the taper is new: the 2014 wave did not vanish at the ends of the piece, so the last driven point stood up to 4 cm proud of the first free point of the wall one spacing away, and the run aborted at $t = 0.77$ s (slide 37);
- `Lap` $= 0.3$ m is the wavelength; the 2014 file called it `Let`, which in the generator is the elastic tube's length (0.4 m), a name clash IB/11 removes;
- the amplitude is a position, so this mode has no F_{mag} to tune and no risk of crossing walls, and the force it needs is whatever the tethers must supply.

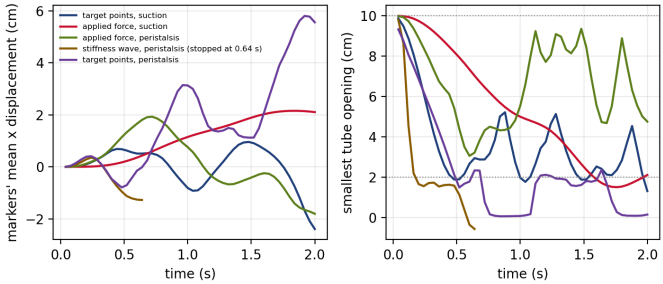
THE FIVE MODES IN ONE TABLE

mode	driving structure	what is set each step	force law	what is prescribed
`TARGET_VP`	`actuator_top/bot` (2 structures)	target positions	tether (target point)	the walls' positions at one spot
`FORCE_VP`	`vp_aforce`	`params[1]` = phase	index 1: F_{mag}·phase	a force across the tube at one spot
`FORCE_PERI`	`peri_aforce`	`params[1]` = travelling Gaussian	index 2: F_{mag}·phase	a travelling force
`STIFFNESS_PERI`	`peri_kwave`	`params[0]` = travelling stiffness	index 0: $k(x,t)\,(R-0.05)$	a travelling pull toward 50% opening
`TARGET_PERI`	`pperi_top/bot` (2 structures)	target positions	tether	a travelling wall shape

The same tube, pericardium, racetrack and markers in all five; the same `main.cpp`, which dispatches on `ACTUATION`; one input file each. The next slides show what the five do in two periods.

Run times are in the README; four modes run at `DT = 5e-5` (the course's `1e-5` was a safe choice for a 10 s run; chapter 8a's rule puts the limit between the two for these stiffnesses), and `FORCE_PERI`, on its ten-times-stiffer tube, at `2.5e-5`.

WHAT THE FIVE MODES DO: TRANSPORT AND SQUEEZE



Left: the mean x-position of the 121 markers, which start inside the elastic section, against time: how far those parcels drift along the tube, which is not always the direction of the net flow round the loop (next slide). Right: the smallest opening of the elastic section against time: how hard each mode squeezes (the diameter is 10 cm; the suction targets close to 2 cm by construction, and the 2014 target wave closes the tube and reopens it).

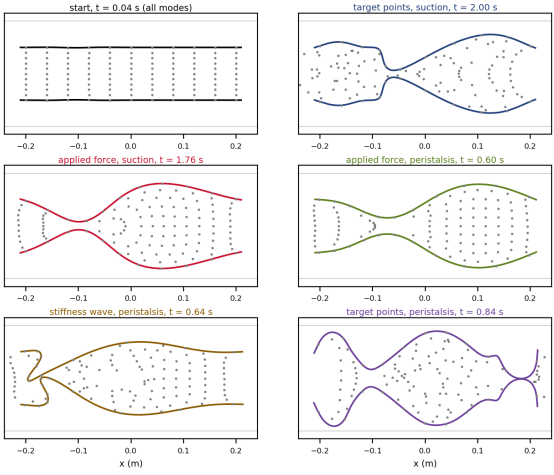
The numbers are in the README and on slide 29 (two periods, shipped parameters; a curve that ends early ends where its run stopped). What to look for: next slide.

IB/11 with IBAMR 0.19.0, the five input files as shipped, 2 s each; `tools/ch9b_figures.py` reads `hier_data_IB2d/X.*` (the structures in input order; the markers last).

WHAT THE FIVE MODES DO: WHAT TO LOOK FOR

- The target-driven modes squeeze as told, and the elastic walls between the tethered copies go a little further.
- The applied-force modes squeeze as far as F_{mag} can push against the tube and the fluid, and nothing tells them when to stop (slide 28).
- The markers' drift has a direction, and it depends on the squeeze: the target-driven suction pump sends the markers toward the nearer end of the elastic section (-2.4 cm in two periods); the force-driven one, which squeezes less and later, sends them the other way ($+2.1$ cm); the target wave sends them with the wave ($+5.6$ cm); the force-driven wave on the stiff tube sloshes them back and forth ($+0.8$, then -2.6 cm). The net flow round the loop, integrated across the tube from the velocity (chapter 29), turns the same way in both suction modes: clockwise, $-x$ along the elastic section, 1.6×10^{-3} m^2/s with the targets and 6.7×10^{-4} with the applied force (two-period means); the force-driven markers drift against it.
- The target-driven suction pump's smallest opening in its second period (1.3 cm at $t = 2$ s) is beside the actuator, not in it: the actuator has reopened and sucks the neighbouring elastic wall in, which is what "dynamic suction" means (slide 27, top right).
- The net flow per period is the figure of merit of a pump; measure it from the velocity across the tube (chapter 29). It depends on the wall motion achieved and its timing, not on how the motion was produced: the same wall motion gives the same flow, and the five modes achieve five different motions (slide 29).

WHAT THE FIVE MODES DO: THE TUBE AT FULL SQUEEZE



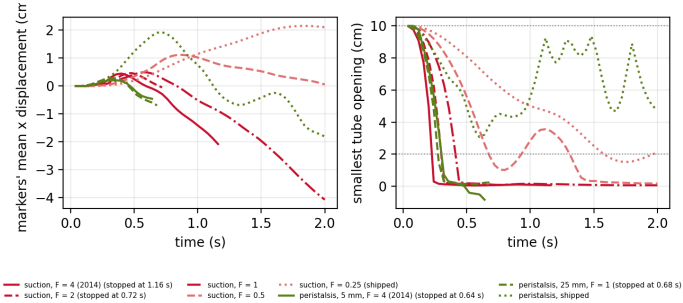
The elastic section (both walls), the pericardium's walls (grey lines) and the markers (grey dots) at the start and at the moment of the smallest opening in each mode.

- the suction modes (1, 2) pinch one spot a quarter of the way along; the elastic walls around the pinch bulge;
- the peristaltic modes (3, 4, 5) show the travelling constriction and, ahead of it, the bulge of fluid it pushes;
- the markers' positions show where those 121 parcels have gone since $t = 0$ (not the net flow, chapter 29); compare the panels with the curves of slide 25.

The 2014 slides showed the same two pictures from VisIt ("VP with applied force", "Results from peristalsis"), with velocity vectors; chapter 26's recipe reproduces them from `viz_IB2d`, and the README has one.

Same runs; `tools/ch9b_figures.py`.

TUNING THE APPLIED FORCE



The two applied-force modes at the 2014 values and at the shipped ones: the markers' transport (left) and the smallest opening (right). Solid: the 2014 code's $F_{\mathrm{mag}} = 4$ (suction) and its 5 mm packet with $F_{\mathrm{mag}} = 4$ (peristalsis); dashed and dotted: smaller forces, the lecture's 25 mm packet, the stiffer tube.

Shipped: the largest forces tried that do not close the tube in two periods, 0.25 for the actuator and 1 for the 25 mm packet on the stiffer tube; every run's numbers are on the next slide.

"You will need to tune it as you change parameters" (2014) is the whole of this slide: an applied force has no notion of the shape it should produce, and the first run with a new F_{mag} is a measurement of what it does.

IB/11, IBAMR 0.19.0; runs that stopped at the CFL limit are marked with the time they reached; `tools/ch9b_figures.py`.

TUNING THE APPLIED FORCE: THE NUMBERS

Every applied-force run of the chapter, from the saved positions. “Closes”: the walls come within two grid cells (0.4 cm; $\Delta x = 0.195$ cm), share cells and stick. Drift: the markers’ mean displacement over 0–1 s and over 1–2 s (not the net flow, chapter 29). Opening: the smallest wall-to-wall distance in each period.

run	tube	closes at	run ends	markers’ drift (cm)	opening (cm)
suction, $F_{\mathrm{mag}} = 4$ (2014)	2014	0.24 s	abort, 1.16 s	-1.4	0.05
suction, 2	2014	0.32 s	abort, 0.72 s	–	0.07
suction, 1	2014	0.44 s	2 s	-0.5, -3.6	0.09, 0.06
suction, 0.5	2014	1.44 s	2 s	$+1.0$, -1.0	1.0, 0.17
suction, 0.25 (shipped)	2014	never	2 s	$+1.1$, $+1.0$	5.0, 1.5
peristalsis, 5 mm, 4 (2014)	2014	0.36 s	abort, 0.64 s	–	crossed
peristalsis, 25 mm, 1	2014	0.32 s	abort, 0.68 s	–	0.13
peristalsis, 25 mm, 1 (shipped)	$10 \times EI$	never	2 s	$+0.8$, -2.6	3.1, 4.7
target points, suction	2014	never	2 s	-0.8, -1.6	1.9, 1.3
target points, peristalsis	2014	0.76 s, reopens	2 s	$+3.1$, $+2.4$	0.07, 0.09
stiffness wave, 25 mm, rest 5 cm	2014	0.56 s	abort, 0.64 s	–	crossed

Two things to read off. The shipped suction force pumps steadily (its net flow never reverses) and turns the loop the same way as the target-driven pump, though its markers drift the other way (chapter 29). Its squeeze is shallower and lags the force by half a period (5.0 cm at $t = 1.0$ s for a force that peaked at 0.5 s). The force-driven wave on the stiff tube runs but sloshes: the fluid it pushes ahead flows back when the packet leaves the piece, and two periods do not settle its direction; a larger F_{mag} on the stiff tube is the first thing to try (slide 38).

Same runs; `tools/ch9b_figures.py` prints these numbers.

MEASURING A PUMP: THE PUMP COEFFICIENT

From the 2014 lecture, “Analysis – pump coefficient”: “The pump coefficient is given by $C_{\mathrm{pump}} = m\,U_a^2/W$, where m is the total mass of fluid in the loop, U_a is the average fluid velocity in the direction of the loop, and W is the total work done by the pump over a set period of time. Note that work is $F \cdot d$, and we calculate each of these in IBM.”

- $m = \rho\,A_{\mathrm{loop}}$ per unit depth, from the geometry;
- U_a from the velocity field integrated across the tube: the flux divided by the diameter (chapter 29); not from the markers’ displacement, which can differ even in sign (slide 26);
- $W = \sum_{\text{steps}} \sum_{\text{driven points}} \mathbf{F}_k \cdot \Delta\mathbf{X}_k$: the force on each driven point times its displacement, summed over a period, from the `F.*` and `X.*` files (chapter 11). For the target modes the force is the tether force; for the force modes it is F_{mag}·phase on each spring, which `F.*` also contains.

> C_{pump} is a kinetic energy over a work: the fraction of the pump’s effort that ends up moving the whole loop. It is one of chapter 8b’s dimensionless outputs, and the quantity the valveless-pump studies of Part VI compare across frequencies, Wo and actuator positions.

OTHER LAWS YOU CAN WRITE

```
// a spring that only pulls (a cable, a tendon): slack under compression
inline double cable_force(double R, const double* params, int, int)
{   const double k = params[0], r0 = params[1];
    return (R > r0) ? k * (R - r0) : 0.0; }
// a stiffening (cubic) tissue spring: three numbers per line of the .spring file
inline double cubic_force(double R, const double* params, int, int)
{   const double k = params[0], r0 = params[1], k3 = params[2], e = R - r0;
    return k * e + k3 * e * e * e; }
// a muscle: passive spring plus an active tension that the update function writes into params[2]
inline double muscle_force(double R, const double* params, int, int)
{   const double k = params[0], r0 = params[1], T_active = params[2];
    return k * (R - r0) + T_active; }
```

The pattern is always the same: a tension from R and the parameters; anything that varies in time or along the fibre arrives through a parameter that the update function writes.

- a force–length muscle law: make `T_active` depend on R too, e.g. a Gaussian in R/r_0 about its optimum;
- a force–velocity law needs the velocity, which a spring function does not get; use a Lagrangian force strategy (next slide) or store the previous length in a parameter and difference it;
- IB2d (chapter 4e) ships muscle models of exactly these forms, which is a good place to test a law before writing it in C++;
- give each law its own index and its own generator loop, and write the derivative or pass `nullptr`.

Chapter 10 does the same for beams (a preferred curvature that changes in time makes a fin flap); chapter 36 drives a tube with two such muscles out of phase.

BEYOND SPRINGS: THREE OTHER WAYS TO PUT A FORCE ON A STRUCTURE

```
// 1. A Lagrangian force strategy of your own (IBLagrangianForceStrategy.h): the whole force on every point
class MyForce : public IBLagrangianForceStrategy
{   void computeLagrangianForce(Pointer<LData> F_data, Pointer<LData> X_data, Pointer<LData> U_data,
                                Pointer<PatchHierarchy<NDIM>> hierarchy, int level_number,
                                double data_time, LDataManager* l_data_manager) override;  ... };
// several strategies at once (yours and the standard one): IBLagrangianForceStrategySet(begin, end)
ib_method_ops->registerIBLagrangianForceFunction(new IBLagrangianForceStrategySet(v.begin(), v.end()));
// 2. A uniform body force on one structure (IBStandardForceGen.h); F is an IBTK::Vector
ib_force_fcn->setUniformBodyForce(F, structure_id, level_number);
// 3. An Eulerian body force on the fluid (chapter 7): the ForcingFunction block of the input, or any CartGridFunction
time_integrator->registerBodyForceFunction(f_fcn);
```

- A force strategy is what `IBStandardForceGen` itself is: a class whose `computeLagrangianForce` fills the force vector $\mathbf{F}$ from $\mathbf{X}$, $\mathbf{U}$ and the time. Write one when the force is not between pairs of points: a drag on each point, a force toward a surface, a force–velocity muscle, a feedback controller. It sees the time and the velocities, which spring functions do not.
- `setUniformBodyForce` adds the same vector to every point of a structure: gravity on a dense body (with chapter 13's mass), or a thrust.
- An Eulerian body force acts on the fluid, not on the structure: the stabilizer of chapter 7 is one; a pressure gradient driving a channel is another.

Chapter 15 reads the upstream examples that use these; chapter 36's pump stays with spring functions, which are enough for most muscles.

PREVIEW: THE TWO-POINT VALVELESS PUMP (CHAPTER 36)

The lab's 2019 set-up note (`IBM-SetUp.pptx`), kept word for word, is the next step after IB/11 and the subject of chapter 36 and example IB/12:

- "Domain is $L \times L$, keep actual dimension similar to original simulation. Straight section $= L/2$, $d = L/16$, $dx = L/512$, $ds = L/1024$. Should be 512 boundary points along straight section."
- "Target points: beams and springs. Moving target points to press tube centered at $\frac{1}{4}$ and $\frac{3}{4}$ along length of tube. Clamping regions $L/32$ in length, this would be 16 points across." (With $ds = L/1024$ that is 32 points; the note's 16 counts at dx.)
- "Put target points that move at these locations. They should squeeze inward with a sinusoidal motion. Start with a small amplitude. Add two target points at end of straight tube to hold ends in place."

PREVIEW (2): THE TWO-POINT PUMP'S ONE PARAMETER

- The README of that code: "`update_target_point_positions.C` is a function that is used to move the target points responsible for pumping. The main thing you can change is `static const double phase_lag = 0.0`": the phase between the two pumps, the parameter of the study (Giesbrecht, Miller and Shi, *A theoretical study of two-point dynamic suction pumping*, the lab's poster).
- With two actuators, the net flow depends on their phase difference, their spacing and Wo; with one, as in IB/11, it depends on the actuator's position along the tube, which is why the homework moves it.

> Everything in that set-up is chapter 9a's target points and this chapter's actuator, twice. Chapter 36 ports the code (with the sweep parameters read from the input file, as chapter 9a recommends) and reproduces the poster's figures.

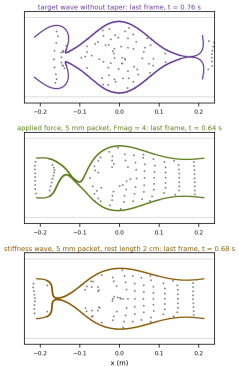
WHAT CAN GO WRONG

The mistake	What happens
the fifth column left out, or 0	the "spring" is a Hookean spring with stiffness F_{mag} and rest length $=$ phase: a stiff spring of rest length 0 to 1 m across the tube; the walls snap together or the run aborts
an index with no registered function	the generator holds a null function pointer for that spring; the run crashes at the first force evaluation (a segmentation fault, no message). Register before `registerIBLagrangianForceFunction`
index 0 overridden by accident	every spring in every file now uses your law, and the tube's own springs with it (slide 9)
the time used inside the force function	it is not an argument; a global set in `main.cpp` works but is invisible to the reader. Put time-dependence in a parameter (slide 11)
the phase set on a copy	the force never changes: chapter 9a's copy bug, in the update function

WHAT CAN GO WRONG (2): THE PUMP

The mistake	What happens
the driving structure not first in `structure_names`	the update function moves or sets the wrong structure, usually the tube's own points (slide 15)
F_{mag} too large	the walls meet and cross; "the contraction may be too much". Reduce it, or use the stiffness wave (slide 22)
a derivative that is wrong for the law	nothing in an explicit run; a slow or failing Newton solve in an implicit one. Pass `nullptr` when unsure
a wrong parameter count	a line with fewer numbers than the law reads gives garbage from memory; more are ignored. Keep every line of a structure the same length
a prescribed wave that does not vanish at the ends of its piece	the last driven point and the first free point, one spacing apart, are torn apart; the beams answer with a huge force and the run hits the CFL limit (next slide)
the two walls driven into the same grid cell	they stick: the IB method cannot pull two surfaces apart once they share a cell, the pinch stays shut, and the fluid piles up ahead of it (next slide)

WHAT CAN GO WRONG (3): THE THREE PERISTALSIS MODES WITH THE 2014 PARAMETERS



All three stopped between $t = 0.67$ and 0.77 s with `Time step size change encountered` (from the IB integrator's `preprocessIntegrateHierarchy`), chapter 16's CFL abort: somewhere the velocity exceeded $0.3\,\Delta x/\Delta t \approx 12$ m/s.

- Target wave, no taper (top): the 2014 wave had a full wavelength on the piece, so its ends moved with it; at $t = 0.75$ s they stood 3.6 cm proud of the free wall one point away (the folds at $x \approx \pm 0.14$). Fixed by the taper of slide 23.
- Applied force, 5 mm packet, $F_{\mathrm{mag}} = 4$ (middle): the first pinch closed the tube at $t = 0.36$ s and the walls crossed; the pinch never reopened, and the fluid the packet pushed piled up in a bulge that reached the pericardium.
- Stiffness wave, 5 mm packet, rest length 2 cm (bottom): the walls overshot the rest length to 4 mm apart, two grid cells, and stuck; then as above.

The 2014 code's ad-hoc penalty term was an attempt at the middle case; IB/11 ships a tapered target wave, a wider packet, a gentler force, a longer rest length and, for the applied-force wave, a stiffer tube instead (slides 20, 22, 28). Tapering the packets themselves was tried and did not help: the collapse is behind the wave, not at the piece's ends.

IB/11 with the 2014 update functions and parameters, IBAMR 0.19.0, `DT = 5e-5`; last saved frame of each run; `tools/ch9b_figures.py`.

HOMEWORK

From the 2013–2015 courses and this chapter (Appendix C collects the full set):

1. IB/11: run `TARGET_VP` and `FORCE_VP`, and find the F_{mag} at which the applied force gives the same squeeze as the targets. Compare the net flow per period (chapter 29) and the markers' drift.
2. IB/11: halve Wo (the viscosity doubles) and halve the frequency, one at a time; say which changes the net flow per period more, which the markers' drift, and why (chapter 8b).
3. Move the actuator to the middle of the elastic section (`Ca` in the generator). What happens to the net flow, and what does that say about valveless pumping?
4. Write a cable (tension-only) law, register it under index 3, and give it to the springs of IB/01's rubber band. What does the band do when it is compressed?
5. Write a muscle law with an active tension that the update function raises and lowers at 1 Hz on half of IB/06's fibre.

> What to hand in: the force function and its registration line, the generator loop that writes the index, the `integral{F}` or marker check that shows the force acts, and one VisIt frame per run.

Extension: compute C_{pump} (slide 30) for the two suction modes from the `X.*` and `F.*` files.

RULES OF THUMB

- A spring function returns the tension from R and the parameters; positive pulls together. No time, no velocity: those come through parameters or a force strategy.
- Register under an index other than 0 unless you mean to change every spring; write the index in the fifth column; register before the force generator is handed to the IB method.
- Parameters are yours: a force magnitude and a phase are as legal as a stiffness and a rest length. Say what they are in the generator.
- Time-dependence: an update function sets a parameter through a reference before each step.
- Applied forces can overdo it; a stiffness wave with a rest length cannot; a target wave is kinematics.
- The driving structure first in `structure_names`; one input file per mode.
- Derivative: exact if you know it, `nullptr` if not, never a guess.
- Check with `integral{F}`, the markers, and one frame in VisIt.

SUMMARY

1. A spring in IBAMR is a tension law $T(R, \text{params})$ along the line between two points, chosen per spring by an index; the default is Hooke's law, and your own is registered with `registerSpringForceFunction(index, &f, &df)`.
2. The 2013 interface (a force vector from a displacement vector and two named parameters) became, in 2015, a tension from R and a parameter array, with an optional derivative; old code ports in a few lines.
3. Time enters through parameters that an update function sets each step: a phase for an applied force, a stiffness for a travelling wave. The heart-tube pump does this five ways, and the same `main.cpp` runs them all.
4. The applied-force modes trade the certainty of target points for a force the fluid can answer; the stiffness wave bounds the squeeze by its rest length; the pump coefficient measures what each achieves.
5. For forces that are not between pairs of points there are force strategies, uniform body forces and Eulerian forcing; the two-point pump of chapter 36 needs none of them.

Next, chapter 10: resting curvature and beams, where the preferred curvature of a beam becomes the third kind of prescribed motion.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The interface is quoted from 0.19.0 and the 2013 one is kept for reading old code; the header's real name is given (`IBSpringForceFunctions.h`, not `IBSpringFunction.h`) and the default function's (`default_spring_force`).
- The 2014 lecture's code is fixed: the undefined `c`, the loop over departed packets, the index-2 registration, the rest-length slot reused as a phase (now documented), and the structure-order dependency (now stated and checked by the mode switch).
- The three peristalsis modes were run with the 2014 parameters first and all three aborted at the CFL limit before 0.8 s (slide 37); the prescribed wave now tapers at the ends of its piece, the packet's width and speed are input keys, and the shipped values are the ones that run two periods.
- The example is assembled from the lab's 2014 `Forced-tube-pump` code and the lecture's version of the peristaltic force, with five modes selectable from the input file instead of comments in `main.C`; the 2015 public-folder example, which lacked input and structure files, is subsumed. The pumping frequency is read once from the input.
- The 2015 slides' "new addition highlighted in red" is stated in words, since the highlighting was lost.
- New: the zero-derivative explanation, the fifth-column rules from the initializer, the five-mode table, the transport and squeeze figures, the pump coefficient's recipe, the other laws and the other three force mechanisms, the two-point pump preview, and the what-can-go-wrong table.
- The 2014 `main.C`'s penalty term in the peristaltic force is described but not shipped; the stiffness-wave version it ran is shipped as a mode of its own.

REFERENCES

- IBAMR 0.19.0: `include/ibamr/IBSpringForceFunctions.h` (the interface and the default law), `include/ibamr/IBStandardForceGen.h` and `src/IB/IBStandardForceGen.cpp` (registration, the force loop, the finite-difference derivative), `include/ibamr/IBLagrangianForceStrategy.h`, `IBLagrangianForceStrategySet.h`, `IBMethod.h`; `examples/IB/explicit/ex3/example.cpp`. https://github.com/IBAMR/IBAMR
- The example of this series: IB/11 heart-tube pump (`main.cpp`, five `update_*.cpp` files, `pump_geometry.h`, `generate_mesh2d.py`/`.m`, `make_inputs.py`, five input files, README with the measured results); `tools/ch9b_figures.py`.
- The lab's 2014 `Forced-tube-pump` code (IBAMR-Examples repository) and the 2014 working folder's `notes-gaus-wave` note, the sources of IB/11.
- K. Giesbrecht, L. A. Miller and J. Shi, *A theoretical study of two-point dynamic suction pumping* (poster, 2019–2020); the 2019 two-point pump set-up (`IBM-SetUp.pptx`, README), the sources of chapter 36.
- Sources: *Creating your own spring function in IBAMR* (MATH 769, October 2013); *Designing your own spring force function: examples for peristalsis and dynamic suction pumping* (June 2014, 37 slides); *IBAMR: Creating Spring Force Functions* (A. Hoover, 2015; public folder 14, and 15 "IBAMR Miscellaneous").